1. Descripción general

La aplicación es un programa de escritorio hecho en Python con PyQt5 que administra una agenda de contactos guardada en un archivo de registros de tamaño fijo (contactos.dat). Permite llegar directamente a cualquier contacto con seek(), usa un índice en JSON (indice.json) para buscar por ID, exporta a JSON y PDF, verifica la integridad con SHA-256 y crea respaldos comprimidos en ZIP. Todos los archivos se leen y escriben en UTF-8.

El código está dividido en dos módulos: almacenamiento.py(toda la lógica de archivos) y main.py(la interfaz con barra lateral: Contactos, Exportar, Herramientas y Ayuda).

2. Estructura del registro

Cada contacto ocupa exactamente 100 bytes. Cada campo se rellena con espacios hasta su largo máximo (función _ajustar) y el registro termina con un salto de línea, de modo que el archivo también se puede abrir como texto.

Campo	Largo (bytes)	Posición en el registro
id	6	**0 - 5**
nombre	30	**6 - 35**
telefono	12	**36 - 47**
correo	32	**48 - 79**
ciudad	19	**80 - 98**
salto de línea	1	**99**
TOTAL	**100**	**0 - 99**

Los largos se miden en **bytes** y no en caracteres, porque una letra con tilde en UTF-8 ocupa 2 bytes. Así todos los registros miden siempre 100 bytes y la fórmula de posición es exacta.

Posición de un registro = número de registro x 100. Por ejemplo, el registro #24 empieza en el byte 2400.

3. Acceso aleatorio con seek() y tell()

Para leer o modificar un contacto nunca se recorre el archivo: se calcula su posición y se salta directamente a ella con `seek()`. Con `tell()` se consulta la posición actual, y la interfaz la muestra en la consola inferior después de cada operacion.

Lectura (buscar por ID)

```
numero = self.indice[id_]                  # 1) el indice dice en que registro esta with
open(ARCHIVO_DATOS, "rb") as f:
    f.seek(numero * TAM_REGISTRO)          # 2) salto directo al registro
    antes = f.tell()                       #    posicion antes de leer
    registro = f.read(TAM_REGISTRO)        # 3) se lee SOLO ese registro (100 bytes)
    despues = f.tell()                     #    posicion despues de leer
```

Modificación en el lugar

```
with open(ARCHIVO_DATOS, "r+b") as f:      # r+b: leer y escribir sin borrar el archivo
    f.seek(numero * TAM_REGISTRO)
    f.write(empaquetar(c))                 # solo se sobrescriben esos 100 bytes
    despues = f.tell()
```

Agregar un contacto

```
with open(ARCHIVO_DATOS, "r+b") as f:
    f.seek(0, 2)                           # ir al final del archivo
    fin = f.tell()                         # tell() = tamano actual
    numero = fin // TAM_REGISTRO           # el nuevo registro es el siguiente
    f.seek(numero * TAM_REGISTRO)
    f.write(empaquetar(c))
```

Ejemplo real tomado de la consola de la aplicación (50 contactos, buscar el ID 25):

```
BUSCAR id=25 -> indice dice registro #24 | seek(2400) | tell() = 2400 -> 2500 MODIFICAR id=25 ->
registro #24 | seek(2400) | tell() = 2500
```

Antes de leer, tell()marca 2400 (inicio del registro #24) y después de leer marca 2500, es decir, se avanzaron exactamente 100 bytes. Al modificar solo se tocan esos 100 bytes; el resto del archivo no se reescribe.

4. Índice primario

El índice es el archivo indice.json, un diccionario que relaciona cada id con su número de registro. Se carga a memoria al iniciar y se guarda de nuevo cada vez que se agrega un contacto.

```
{
    "1": 0,
    "2": 1,
    "3": 2,
    "4": 3,
```

"5": 4,
...

Al buscar por ID, primero se consulta el índice (un acceso directo a un diccionario) y después se hace seek()al registro indicado.

Por qué es más eficiente que una búsqueda secuencial

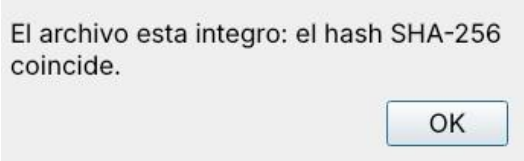
En una búsqueda secuencial hay que leer registro por registro hasta encontrar el ID; en el peor caso se leen los *n* registros. Con el índice siempre se hace una sola lectura de 100 bytes, sin importar cuantos contactos haya.

Contactos en el archivo	Búsqueda secuencial (peor caso)	Búsqueda con índice
50	50 lecturas (5,000 bytes)	**1 lectura (100 bytes)**
10,000	10,000 lecturas (1 MB)	**1 lectura (100 bytes)**
1,000,000	**1,000,000 lecturas (100 MB)**	**1 lectura (100 bytes)**

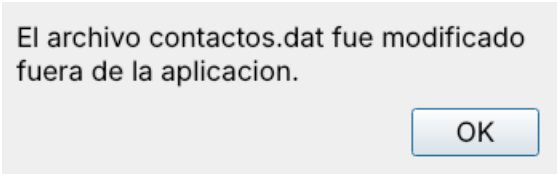
El costo del índice es poco espacio extra (un par id/registro por contacto) y mantenerlo actualizado. Si indice.jsonfalta o está corrupto, la aplicación lo reconstruye leyendo el archivo una vez.

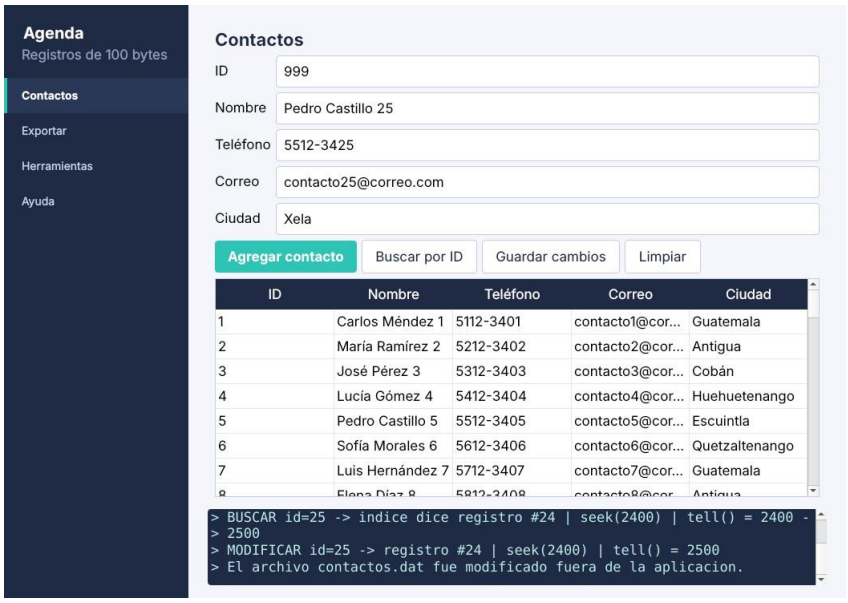
5. Verificación de integridad (SHA-256)

Cada vez que se guarda un cambio se calcula el hash SHA-256 de contactos.daty se guarda en contactos.sha256. Al iniciar la aplicación (y desde Herramientas) se recalcula y se compara.



Para probar la detección se modifico manualmente el archivo, cambiando 8 bytes dentro del registro #5 fuera de la aplicación. Al verificar, el hash ya no coincide y la aplicación avisa al usuario.





6. Respaldo comprimido (ZIP)

La opcion *Crear respaldo ZIP* usa el módulo zipfilecon compresión DEFLATE para guardar contactos.date indice.jsonen un solo archivo, y muestra los tamaños al usuario.

)

Archivo	Tamaño (bytes)
contactos.dat	5,000
indice.json	583
Total original	5,583
Respaldo ZIP (respaldo_20261108_214926.zip)	1,269
Reducción	77.3 % (el ZIP pesa 22.7 % del original)

La compresión es muy buena porque los registros de tamaño fijo contienen muchos espacios de relleno repetidos, que DEFLATE comprime con facilidad. Con pocos contactos el ZIP puede ser incluso más grande que el original, por los encabezados propios del formato.



7. Capturas de pantalla del funcionamiento

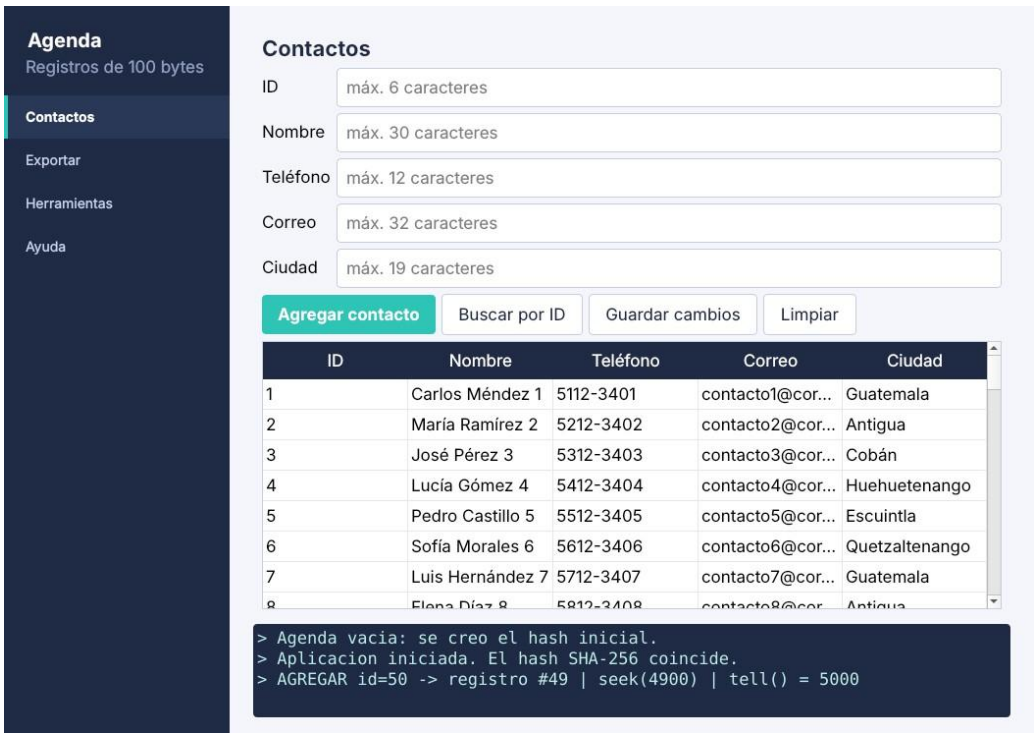
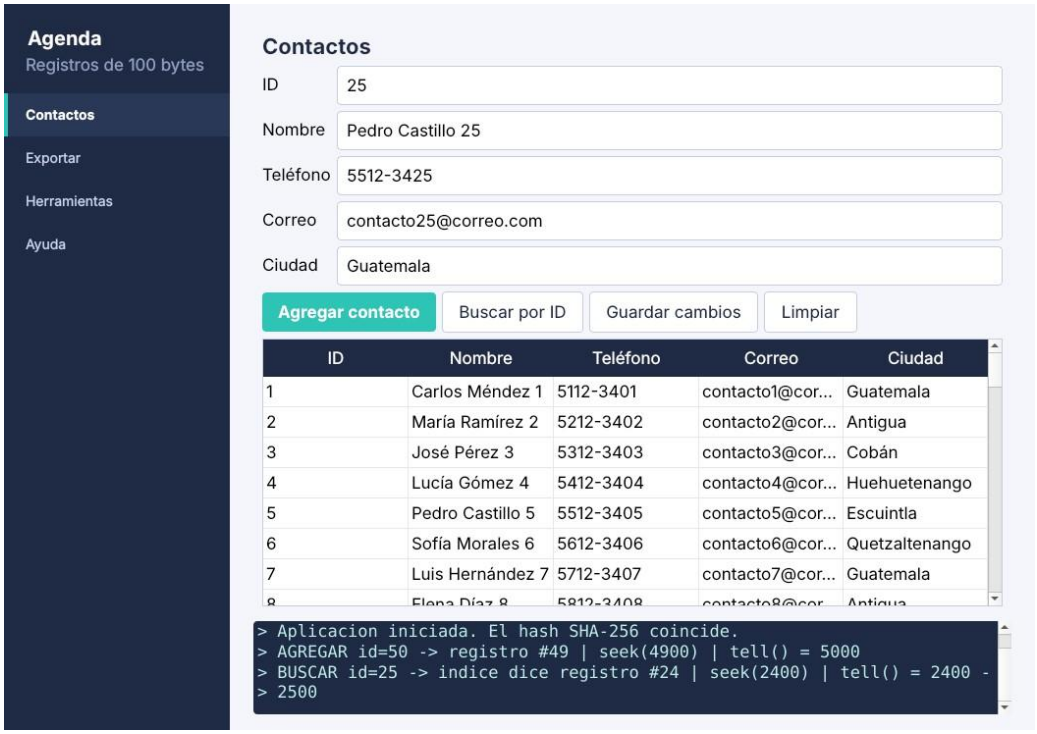
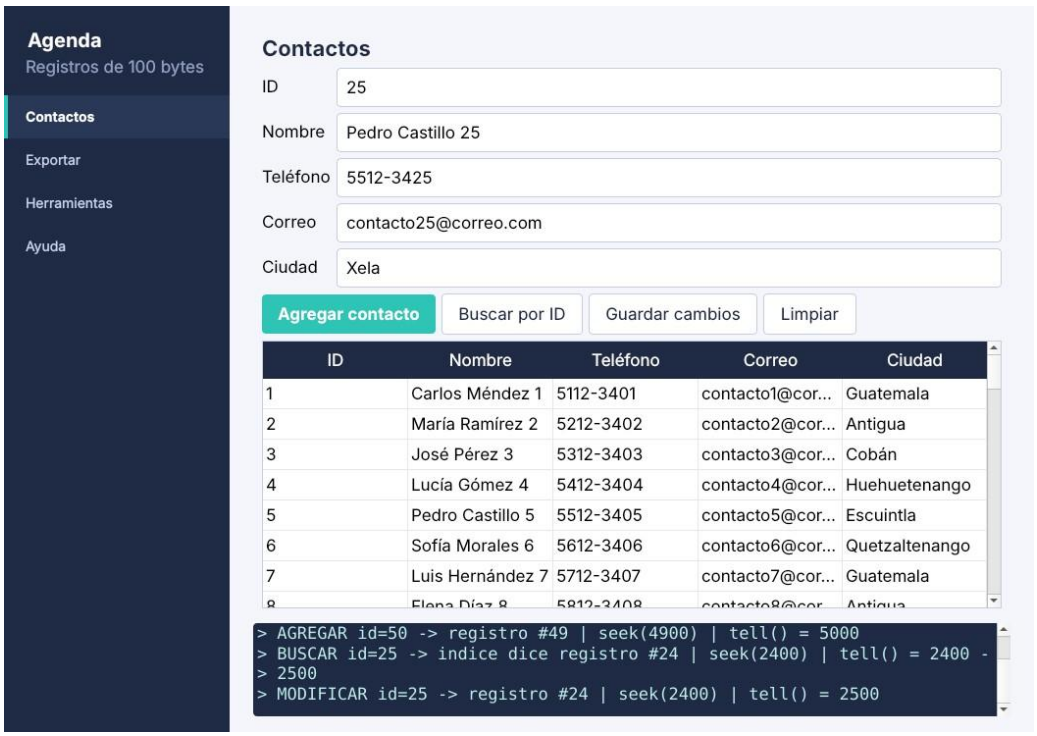
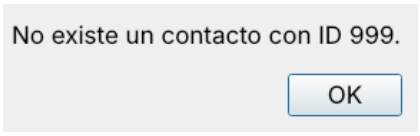
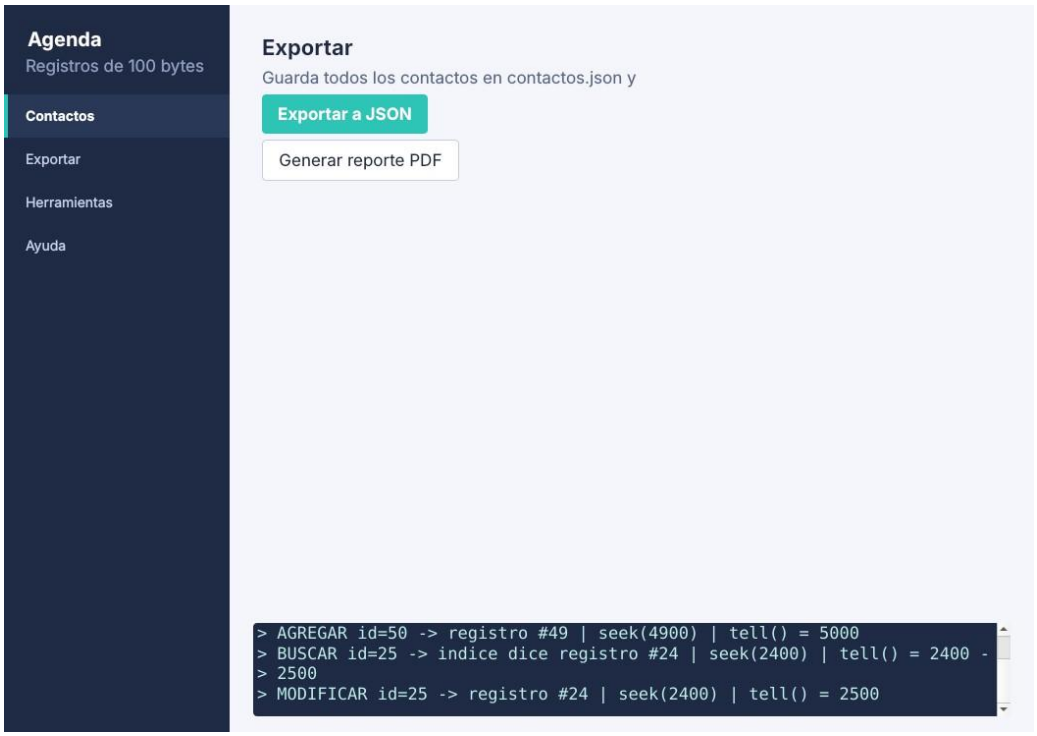


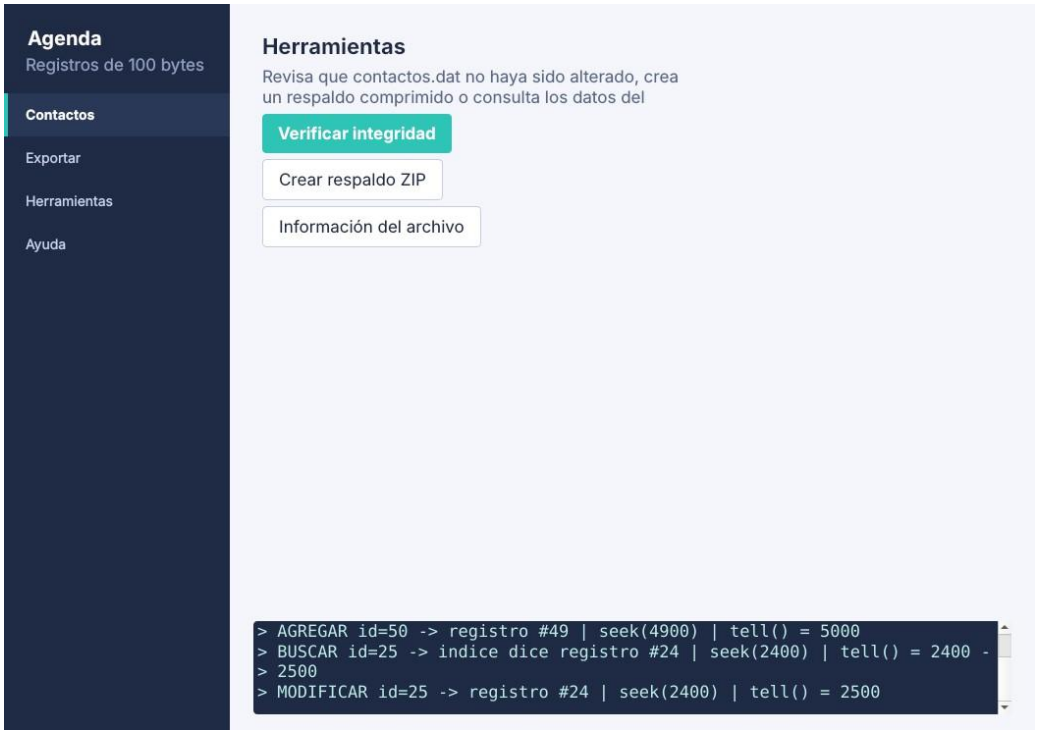
Figura 5. Pantalla de Contactos con la tabla de registros y la consola de operaciones.

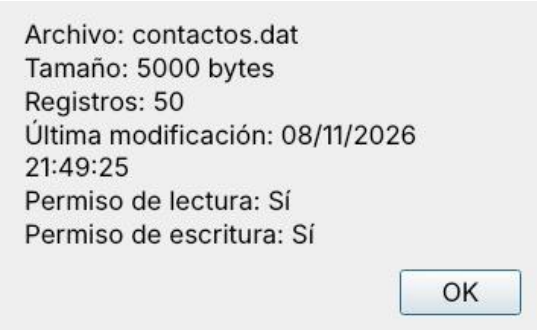

Agenda
Registros de 100 bytes
Contactos
Exportar
Herramientas
Ayuda
Contactos
ID 25
Nombre Pedro Castillo 25
Teléfono 5512-3425
Correo contacto25@correo.com
Ciudad Guatemala
Agregar contacto
Buscar por ID
Guardar cambios
Limpiar
ID Nombre Teléfono Correo Ciudad
1 Carlos Méndez 1 5112-3401 contacto1@cor... Guatemala
2 María Ramírez 2 5212-3402 contacto2@cor... Antigua
3 José Pérez 3 5312-3403 contacto3@cor... Cobán
4 Lucía Gómez 4 5412-3404 contacto4@cor... Huehuetenango
5 Pedro Castillo 5 5512-3405 contacto5@cor... Escuintla
6 Sofía Morales 6 5612-3406 contacto6@cor... Quetzaltenango
7 Luis Hernández 7 5712-3407 contacto7@cor... Guatemala
> Aplicacion iniciada. El hash SHA-256 coincide.
> AGREGAR id=50 -> registro #49 | seek(4900) | tell() = 5000
> BUSCAR id=25 -> indice dice registro #24 | seek(2400) | tell() = 2400 -
> 2500


Agenda
Registros de 100 bytes
Contactos
Exportar
Herramientas
Ayuda
Contactos
ID 25
Nombre Pedro Castillo 25
Teléfono 5512-3425
Correo contacto25@correo.com
Ciudad Xela
Agregar contacto
Buscar por ID
Guardar cambios
Limpiar
ID Nombre Teléfono Correo Ciudad
1 Carlos Méndez 1 5112-3401 contacto1@cor... Guatemala
2 María Ramírez 2 5212-3402 contacto2@cor... Antigua
3 José Pérez 3 5312-3403 contacto3@cor... Cobán
4 Lucía Gómez 4 5412-3404 contacto4@cor... Huehuetenango
5 Pedro Castillo 5 5512-3405 contacto5@cor... Escuintla
6 Sofía Morales 6 5612-3406 contacto6@cor... Quetzaltenango
7 Luis Hernández 7 5712-3407 contacto7@cor... Guatemala
> AGREGAR id=50 -> registro #49 | seek(4900) | tell() = 5000
> BUSCAR id=25 -> indice dice registro #24 | seek(2400) | tell() = 2400 -
> 2500
> MODIFICAR id=25 -> registro #24 | seek(2400) | tell() = 2500


No existe un contacto con ID 999.
OK


Agenda
Registros de 100 bytes
Contactos
Exportar
Herramientas
Ayuda
Exportar
Guarda todos los contactos en contactos.json y
Exportar a JSON
Generar reporte PDF
> AGREGAR id=50 -> registro #49 | seek(4900) | tell() = 5000
> BUSCAR id=25 -> indice dice registro #24 | seek(2400) | tell() = 2400 -
> 2500
> MODIFICAR id=25 -> registro #24 | seek(2400) | tell() = 2500





8. Manejo de errores y ejecución

La aplicación controla los errores con una excepcion propia (ErrorAgenda) y los muestra en un mensaje sin cerrarse: archivo de contactos o índice inexistente, archivo corrupto, falta de permisos, datos invalidos, ID repetido e ID inexistente.

Para ejecutarla: pip install -r requirements.txty luego python main.py.